

AI BlogNest API - AI-Powered Blog Creation and Management

Description

AI BlogNest API is a secure and modular REST API platform designed to simplify blog creation, content management, and AI-assisted writing. Built with Node.js and Express.js, the application uses MongoDB with Mongoose for efficient data storage, validation, and database interaction.

The platform integrates Google Gemini AI to provide AI-powered blog content generation. Users can submit a topic and category through the API, and the backend sends the request to Gemini to generate relevant and structured blog content with minimal manual effort.

Security is an important part of the application. User registration and login are handled through dedicated authentication APIs, while sensitive configuration such as database credentials and the Gemini API key is maintained through environment variables. The backend also provides structured routing, request handling, database validation, and error handling.

Designed with a modular backend architecture, AI BlogNest API combines traditional blog management with Generative AI to provide a practical, faster, and scalable content creation solution.

Scenario-Based Case Study

Background

A content creator wants to build and manage technical blogs but spends considerable time planning topics, writing content, and maintaining blog records. A centralized API with AI assistance can simplify these activities.

Problem

- Manual blog writing requires significant time and effort.
- Managing blog records and categories can become difficult as content grows.
- Creating structured and relevant content for every topic is time-consuming.
- A backend system is required to connect blog management with AI content generation.
- Users need a simple and reliable API for authentication, blog operations, and AI-assisted writing.

Solution

AI BlogNest API provides a centralized backend that separates API routing, controllers, services, database models, and AI integration. The system accepts user requests, processes

blog operations through REST APIs, connects to MongoDB for persistence, and uses Gemini AI for automated blog generation.

Usage

- **User Usage:** Registers and logs in, provides blog details, requests AI-generated content, and manages blog records through the available APIs.
- **Developer Usage:** Uses REST endpoints through Postman or another API client to test authentication, blog management, database operations, and AI generation.
- **AI Usage:** Receives the requested topic and category through the backend and returns generated blog content to the application.

Outcome

The system reduces manual content creation effort, provides organized API-based blog management, maintains persistent data using MongoDB, and demonstrates practical integration of Generative AI with a Node.js backend.

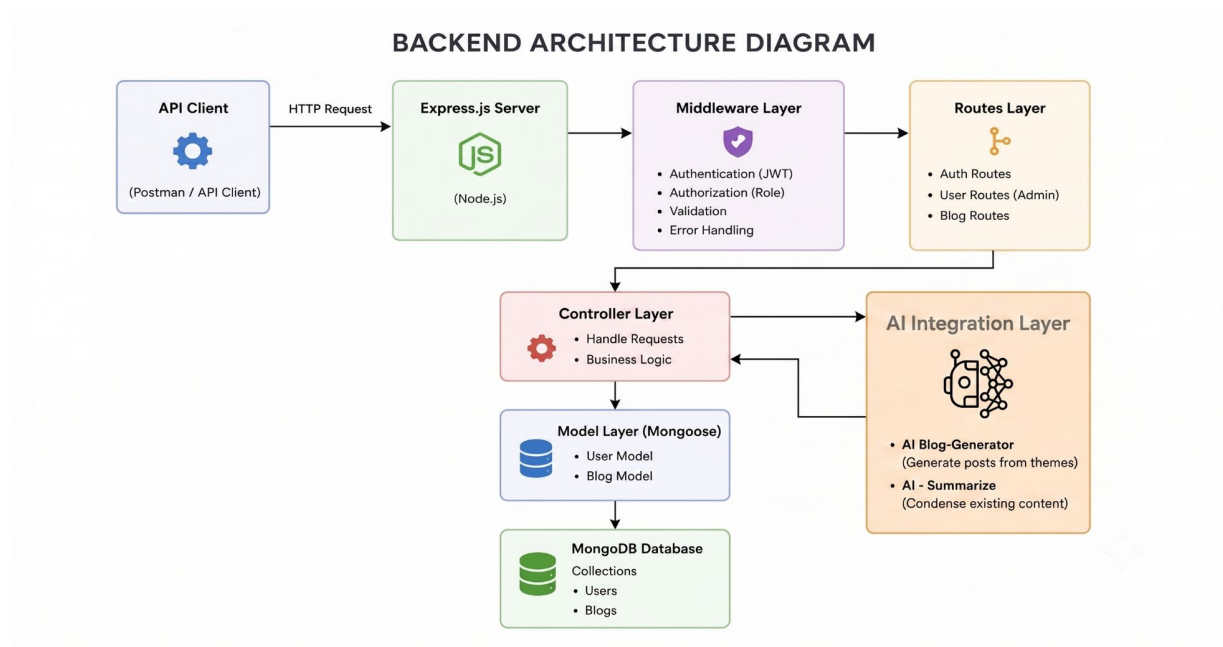
1. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Node.js:** JavaScript runtime used to execute the backend server.
- **npm:** Node package manager used to install and manage project dependencies.
- **Express.js:** Backend framework used to create RESTful API routes and handle HTTP requests.
- **MongoDB:** NoSQL database used to store application and blog data.
- **Postman / Thunder Client:** API testing tools used to send requests and verify responses.
- **Code Editor:** Visual Studio Code or a similar development environment.

2. Hardware Requirements

- **Processor:** Intel Core i3 or above / AMD equivalent.
- **RAM:** 8 GB minimum for comfortable development and database operation.
- **Storage:** At least 1 GB of available space for the project, dependencies, and database tools.

Epic 1: PROJECT ARCHITECTURE



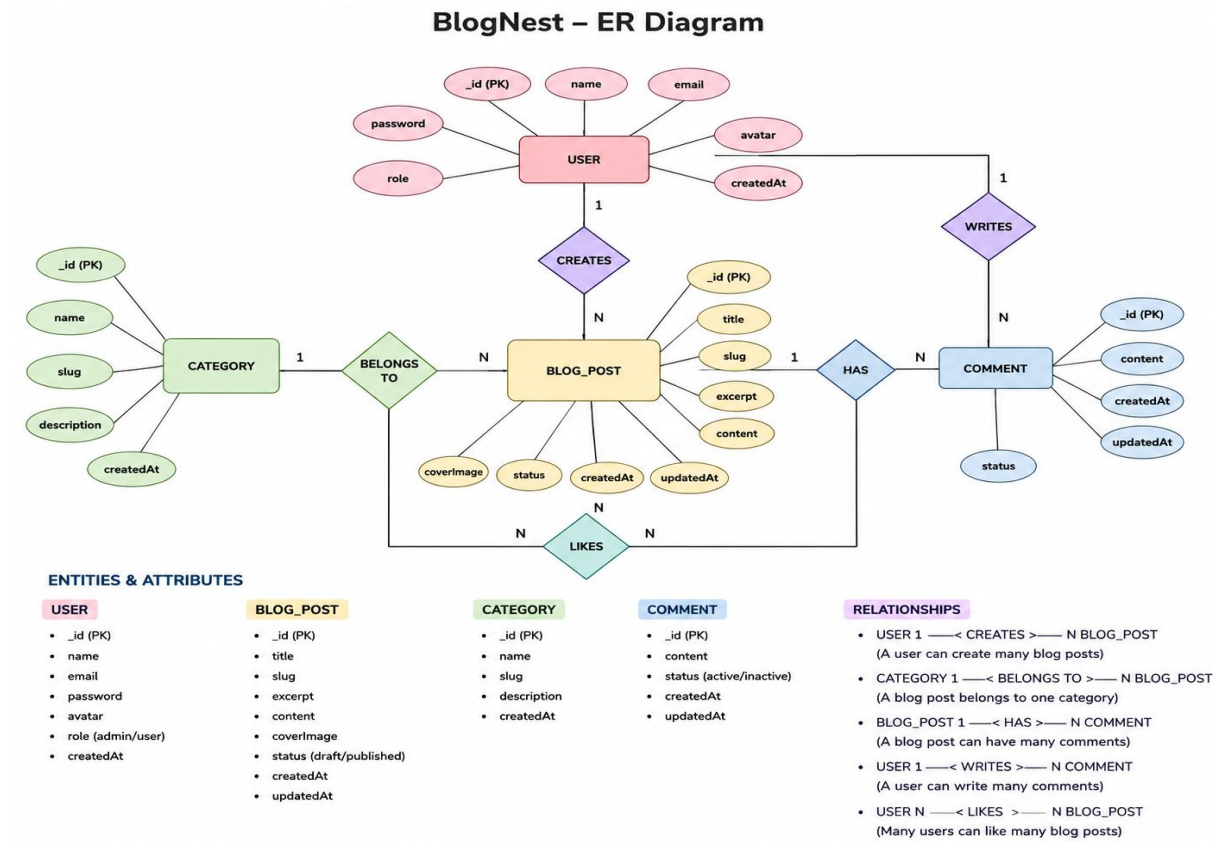
AI BlogNest API uses a layered server architecture in which incoming HTTP requests are received by Express, processed by routes and controllers, and then connected to database or AI services as required.

Components Definition

- **API Interface:** Postman or another API client sends requests to the defined REST endpoints and receives JSON responses.
- **Express Server:** Runs the backend, listens on the configured port, maps HTTP methods, parses JSON requests, and sends API responses.
- **Authentication Middleware:** Protects required routes and verifies the authentication information provided by the client.
- **Controller Layer:** Contains application logic for authentication, blog operations, and AI-powered blog generation.
- **Database Layer:** Mongoose provides structured models and communicates with MongoDB for storing and retrieving application data.
- **AI Service Layer:** Connects the backend with Google Gemini AI through the configured Gemini service and sends prompts for blog content generation.
- **AI Blog Generator:** Receives the topic and category, prepares the AI request, obtains generated content from Gemini, and returns the result to the controller.

Entity-Relationship (ER) Diagram Description

The main application data is organized into logical collections that support authentication and blog management.



1. User Entity

- _id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Stored securely according to the implemented authentication logic)
- authentication details used for user access

2. Blog Entity

- _id (Primary Key - ObjectId)
- userID (Reference to the User Collection, where implemented)
- title / name (String - Blog title)
- category (String - Blog category)

- content / message (String - Blog content)
- additional blog fields such as metadata or timestamps, based on the implemented schema
- timestamps (Used where configured)

Key Structural Mappings

- **User to Blog:** A registered user can create and manage blog records associated with the user account.
- **Authentication to API:** Login provides the authentication information required for protected operations.
- **AI Request to Blog:** A topic and category are passed to the AI service to generate blog content that can be managed through the backend.

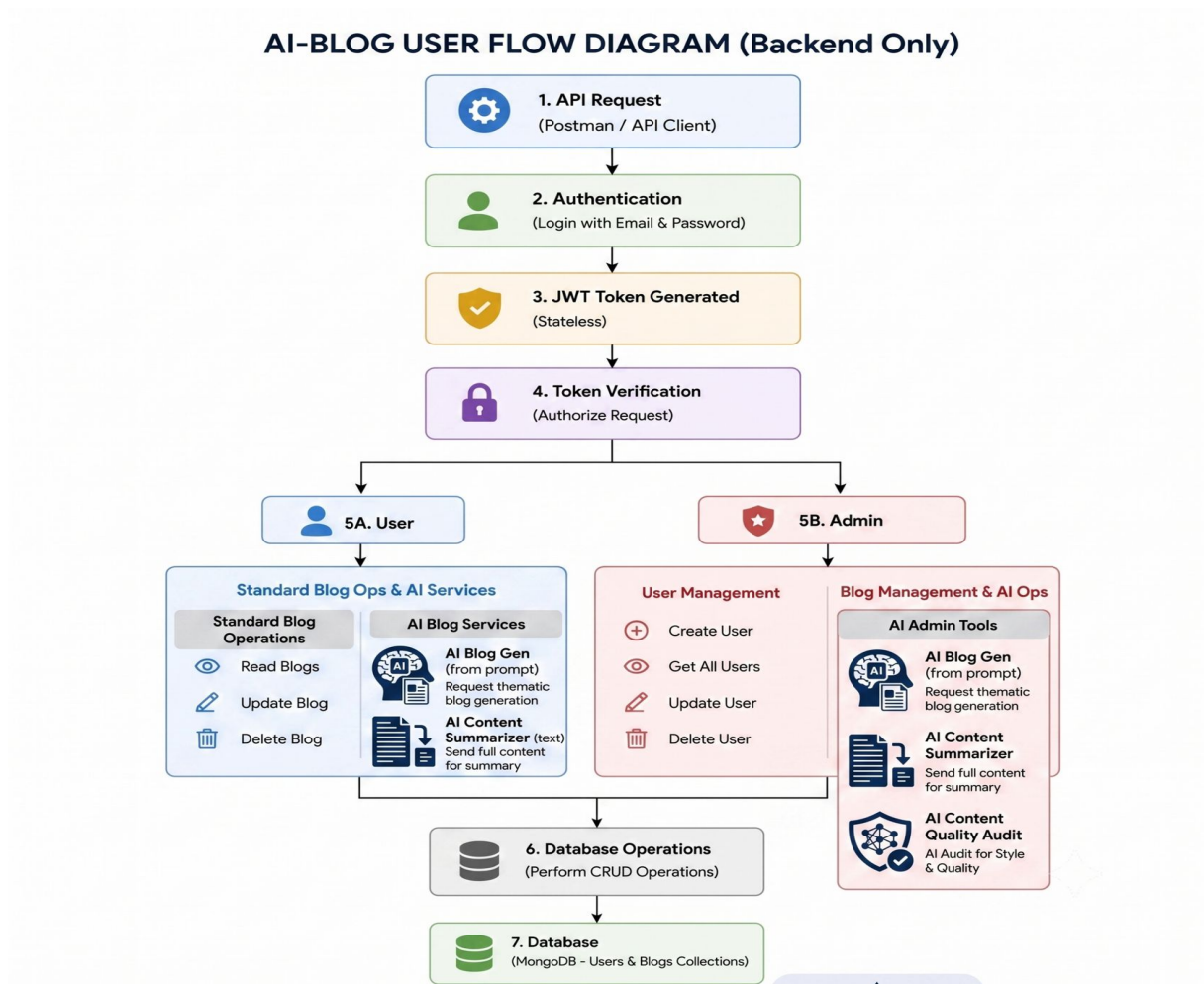
Key Features

- **User Authentication:** Provides registration and login APIs for controlled access.
- **CRUD Blog Operations:** Supports the core create, read, update, and delete workflow for blog records where implemented.
- **Category-Based Content:** Blog requests can include a category to help organize generated content.
- **AI Blog Generation:** Uses Google Gemini AI to generate content from a user-provided topic and category.
- **Environment Configuration:** Keeps sensitive values such as database connection details and API keys outside the source code.
- **Modular Backend:** Separates routes, controllers, services, models, and configuration for easier maintenance.
- **API Error Handling:** Returns structured error responses when validation, database, authentication, or AI-service problems occur.

Roles and Responsibilities

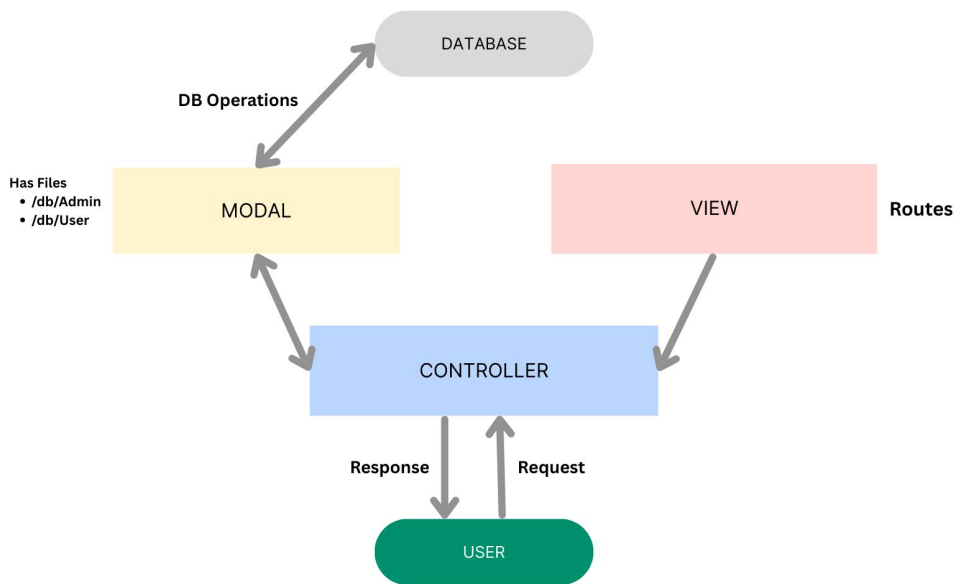
- **Project Administrator:** Maintains the project configuration, database setup, environment variables, and deployment-related settings.
- **Backend Developer:** Develops API routes, controllers, authentication logic, database integration, and server functionality.
- **Database Developer:** Designs MongoDB collections and Mongoose schemas and maintains database connectivity.
- **AI Integration Developer:** Integrates Gemini AI, prepares prompts, handles AI responses, and manages AI-related errors.

User Flow:



MVC Architecture Pattern

The application follows a modular MVC-style backend structure that separates data models, request-processing controllers, and API routing responsibilities.



Model Layer (Data Blueprint)

Defines Mongoose schemas and models used to structure application data and communicate with MongoDB.

Controller Layer (Execution Logic)

Receives requests from routes, validates required information, calls the appropriate services or models, and returns JSON responses.

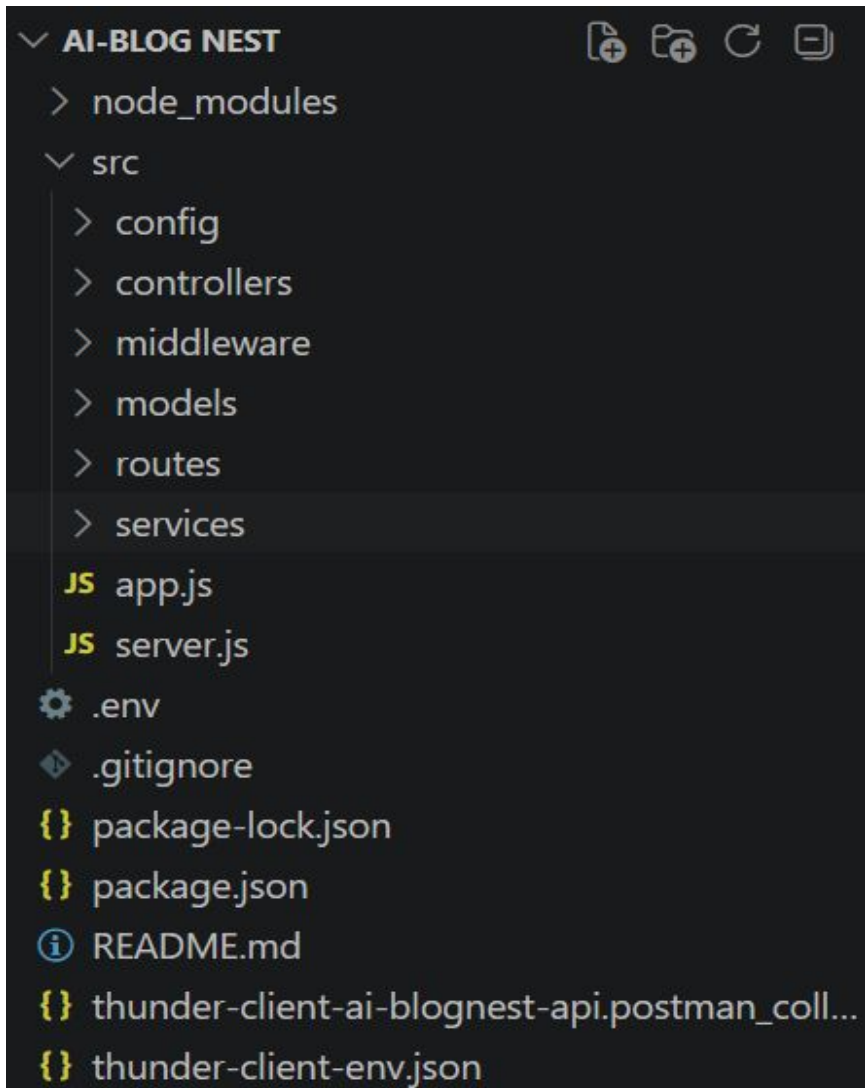
View Layer (API Interface)

As a headless REST API, the project does not require traditional server-rendered views. The API routes act as the interface through which Postman, a frontend, or another client communicates with the backend.

Epic 2: Project Setup and Configuration Creating project folder

1. Create a new project folder named AI BlogNest API.
2. Open the project folder in Visual Studio Code and initialize the Node.js application.

3. Create the required source folders for configuration, controllers, middleware, models, routes, and services. 4. Create the main server and application files.



Server setup (npm init)

- Open the project folder in the VS Code terminal.
 - `npm init -y`
- Create files:
 - `server.js`
- Create folders:
 - `models`

- controllers
- routes

Epic 3: Backend Development Server.js

```
JS server.js ×
src > JS server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10 |   console.log(`Server running on port ${PORT}`);
11 | });
12
```

Configure Express, middleware, routes, database initialization, and server startup.

```
JS app.js ×
src > JS app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22 |   res.json({ message: 'AI BlogNest API is running' });
23 | });
24
25 module.exports = app;
26
```

Epic 4: Database Configuration

DB Connection:

```
Explorer (Ctrl+Shift+E)
src > config > JS db.js > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
22 module.exports = connectDB;
```

1. Configure MongoDB:

- Install Mongoose.

In the Node.js project directory, run:

npm install mongoose

- Create the MongoDB database connection using the configured environment variable.
- Create Schemas & Models.

Create Database Connection:

Make sure MongoDB is connected successfully before performing database operations through the backend. The connection is established from the server configuration and handled through Mongoose.

Mongoose Schema Definitions

1. User and Blog Schemas

Define required fields, validation rules, references, and default values according to the implemented application models.

```
JS User.js ×
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, 'Name is required'],
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: [true, 'Email is required'],
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: [true, 'Password is required'],
20       minlength: 6,
21     },
22   },
23   {
24     timestamps: true,
25   }
26 );
27
28 module.exports = mongoose.model('User', userSchema);
29
```

Epic 5: Project Execution

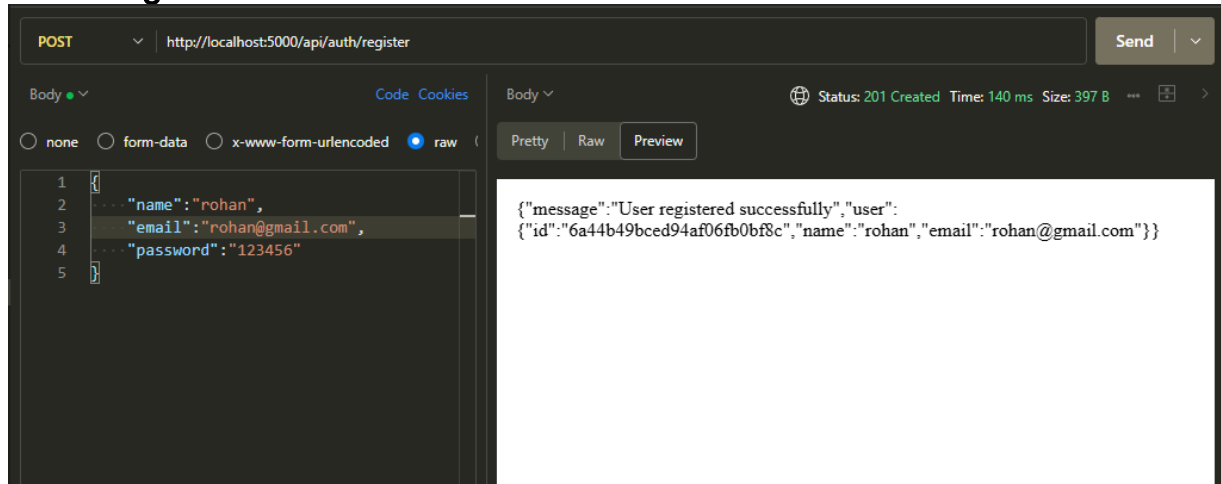
Install Postman or Thunder Client (VS Code extension).

Start the backend server:

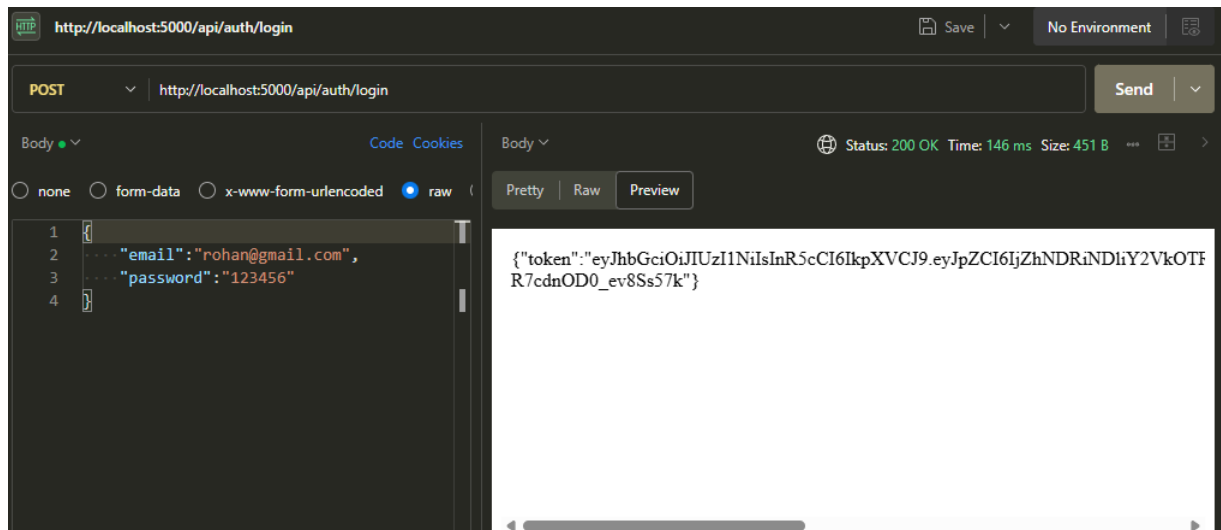
```
npm run dev
```

API Testing

User Registration:



User Login:



Create Blogs:

HTTP **POST** `http://localhost:5000/api/blogs` Save No Environment Send

Headers (11) Code Cookies Body Status: 201 Created Time: 59 ms Size: 541 B

Headers 9 hidden

	Key	Value
☒	Authorization	Bearer eyJhbGciOiUl...
☒	Content-Type	application/json
	Key	Value

Pretty Raw Preview JSON Copy Search

```
1 [{"title": "My First Blog", "content": "This is my first blog content", "category": "Technology", "author": {"_id": "6a44b49bcd94af06fb0bf8c", "authorName": "rohan", "_id": "6a44b6ffcd94af06fb0bf90", "createdAt": "2026-07-01T06:43:11.781Z", "updatedAt": "2026-07-01T06:43:11.781Z", "__v": 0}}
```

Fetch Blogs:

HTTP **GET** `http://localhost:5000/api/blogs` Save No Environment Send

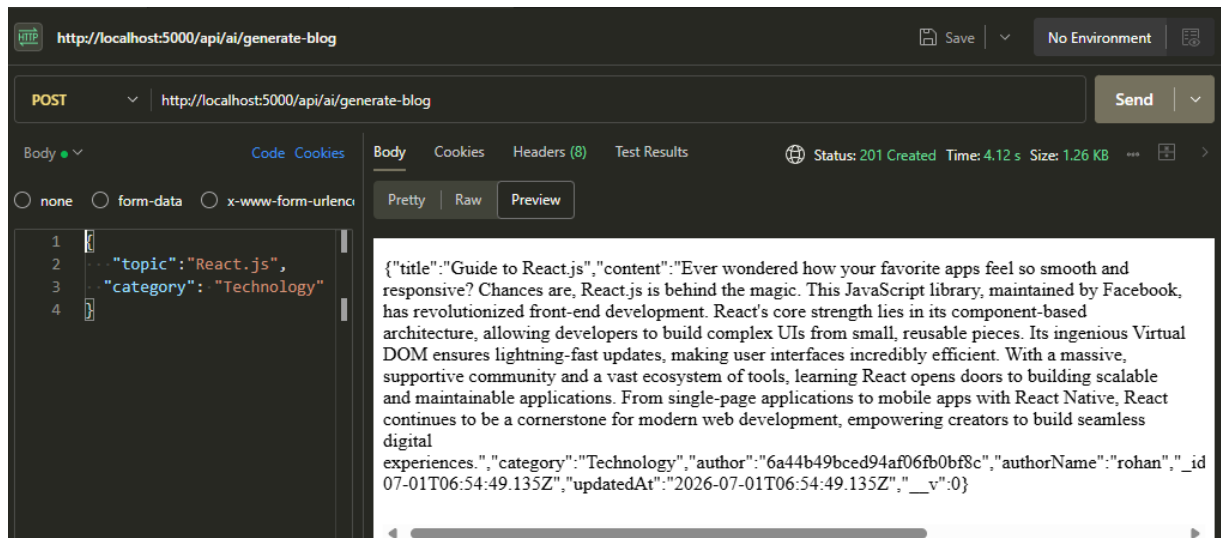
> Body Cookies Headers (8) Test Results Status: 200 OK Time: 92 ms Size: 587 B

Pretty Raw Preview JSON Copy Search

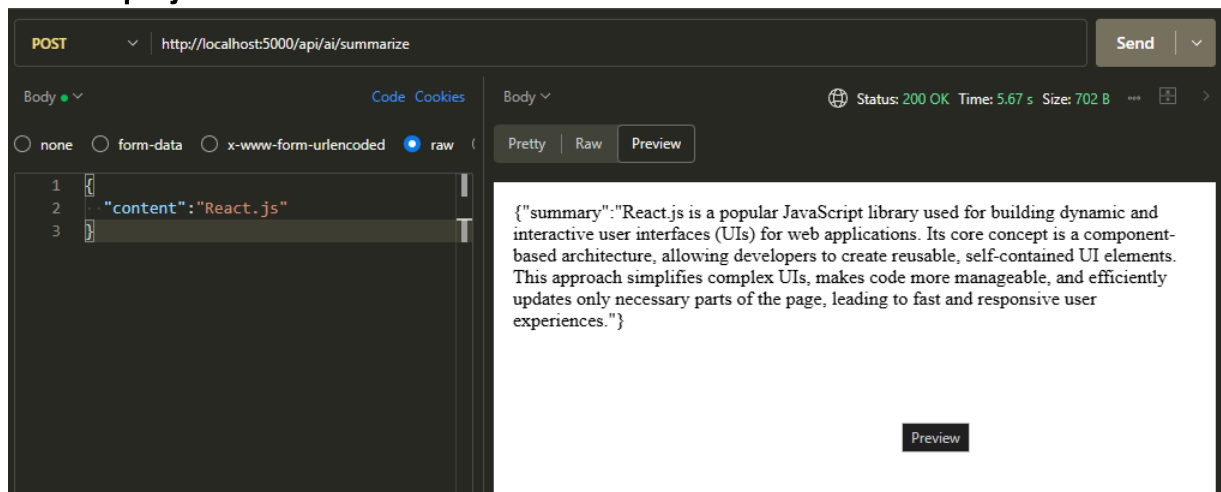
```
1 [{"_id": "6a44b6ffcd94af06fb0bf90", "title": "My First Blog", "content": "This is my first blog content", "category": "Technology", "author": {"_id": "6a44b49bcd94af06fb0bf8c", "name": "rohan", "email": "rohan@gmail.com"}, "authorName": "rohan", "createdAt": "2026-07-01T06:43:11.781Z", "updatedAt": "2026-07-01T06:43:11.781Z", "__v": 0}]
```

Request Editor

Generate AI Blogs:



Summarize Blogs: Use only if the summarization endpoint is implemented in the current project.



Demo Video: <https://drive.google.com/file/d/1ISCPavh6lYpMJXgfOs7Hfouk8Cj8V-ug/view?usp=drivesdk>

API Testing: <https://drive.google.com/file/d/1pSpW61FVwWMurlyPWp8zaS9QEG1kEu/view?usp=drivesdk>

Code Drive: <https://drive.google.com/drive/folders/1BukNHAM3M1uYo1UJpwf7BveyP9-b0U1s>